Find the promos

Which problem I picked and why

I picked Problem 1 because I could check the cuts against the input pack. I wanted to give an editor a short list they could open at the right timecode.

My assumptions

I treat timestamps as episode time. A promo can join adjacent episodes if a speaking character carries across. I aim for 20 to 60 seconds and an ending that leaves something open. I work from screenplays and leave editing to the editor.

How my tool works

Python reads the dialogue, action, camera and sound blocks. It tries windows at event boundaries and blocks cuts inside dialogue. Simple rules score the opening and ending. I cap overlap and lower scores for repeated hook types and speakers.

The optional model scores a shortlist with episode context. A separate call reads only the cut. Code checks both pieces of evidence against the script. An editor can review the ranked cuts in the app and mark the review sheet.

How it scales to every show

I batch four cuts per call and cache valid responses. The default budget takes six requests without retries. With 20 requests a day, I could plan for three shows a day, or 21 a week. This is a quota example.

At an assumed 25,000 input and 3,000 output tokens per show, the paid estimate is about $0.011 per show, or $0.00022 per episode for 50 episodes. I did not measure live usage. I would try a cheap model on a wider shortlist and pay for a stronger model only on finalists.

How I checked quality

The offline run read 50 episodes, scored 103,263 candidates and kept 15 in about 63 seconds. It used zero API tokens. All 15 passed nine hard checks. Codex checked the top 10 against the script. E34, 01:34.20 to 02:06.40, ends just as a judge asks Juliana whether she gave the patients her mixtures. The accusation makes sense inside the cut.

Where it fails

E27, 00:44.50 to 01:32.40, starts with a question about a husband. The doors burst open at 00:48.80. That is too late for the first three seconds. E31 ends on an insult that the scorer treats as an open question.

Tools I used, and one place the AI got it wrong

I used Codex to build and review the Python and Streamlit prototype. Its first scorer called E38 at 00:17.50 a threat because the line says “die.” The guard was giving a prisoner bread. Codex changed the rule after checking the cut. The model flow was tested with a stand-in provider. I had no Gemini key.

What I would do next with more time

I would ask an editor to watch the top cuts, especially the joins, against the footage. I would compare their choices with the offline and Gemini rankings and check which openings actually land in three seconds.